Diseño e Implementación de un Filtro Digital FIR en Python para el Procesamiento de Señales SDR

Bravo Morales Alex Rodrigo, Guaranga Remache Alexis Javier, Pérez Fiallos Priscila Estefania
Carrera de Ingeniería en Telecomunicaciones
Facultad de Ingeniería, Universidad Nacional de Chimborazo
Electrónica II - Grupo 6
alex.bravo@unach.edu.ec, alexis.guaranga@unach.edu.ec, priscila.perez@unach.edu.ec

***Resumen*—Español: Este artículo presenta el diseño e implementación de filtros digitales FIR (Finite Impulse Response) en Python para el procesamiento de señales I/Q capturadas con el dispositivo ADALM-Pluto. Se diseñaron tres tipos de filtros utilizando la librería scipy.signal con ventana de Hamming: pasa-bajas (frecuencia de corte = 100 kHz), pasa-altas (corte = 200 kHz) y pasa-banda (banda = 150-250 kHz), todos con 64 taps. La metodología incluyó la generación de señales multi-tono (50 kHz, 200 kHz y 300 kHz), transmisión y captura con Pluto, aplicación de filtros mediante convolución, y análisis espectral con FFT. Los resultados experimentales muestran una atenuación superior a 40 dB en las bandas de rechazo y una ganancia cercana a 0 dB en las bandas de paso. Se concluye que los filtros FIR diseñados cumplen con las especificaciones técnicas y son efectivos para aplicaciones SDR.**

English: This paper presents the design and implementation of FIR (Finite Impulse Response) digital filters in Python for processing I/Q signals captured with the ADALM-Pluto device. Three types of filters were designed using the scipy.signal library with Hamming window: low-pass (cutoff frequency = 100 kHz), high-pass (cutoff = 200 kHz), and band-pass (passband = 150-250 kHz), all with 64 taps. The methodology included multi-tone signal generation (50 kHz, 200 kHz, and 300 kHz), transmission and capture with Pluto, filter application via convolution, and spectral analysis using FFT. Experimental results show attenuation exceeding 40 dB in stopbands and gain close to 0 dB in passbands. We conclude that the designed FIR filters meet the technical specifications and are effective for SDR applications.

Palabras clave: FIR, Python, SDR, ADALM-Pluto, filtro digital, señales I/Q, scipy.signal.

Keywords: FIR, Python, SDR, ADALM-Pluto, digital filter, I/Q signals, scipy.signal.

I. INTRODUCCIÓN

El procesamiento digital de señales es fundamental en sistemas de radio definida por software (SDR), ya que permite implementar funciones de filtrado, modulación y demodulación mediante algoritmos en lugar de hardware dedicado [**?**]. Los filtros FIR (Finite Impulse Response) son ampliamente utilizados en aplicaciones SDR debido a su estabilidad inherente, fase lineal y facilidad de implementación [**?**].

El dispositivo ADALM-Pluto es una plataforma SDR de bajo costo que permite transmitir y recibir señales en el rango de 325 MHz a 3.8 GHz, siendo ideal para prototipado y enseñanza de conceptos de comunicaciones digitales [**?**]. Este proyecto utiliza el Pluto como generador y capturador de señales para validar el desempeño de filtros FIR diseñados en Python.

Trabajos relacionados como [**?**] y [**?**] presentan los fundamentos teóricos de filtros FIR, mientras que [**?**] aborda implementaciones en Python. Sin embargo, existe una brecha en la integración de estos conocimientos con dispositivos SDR reales como el ADALM-Pluto, especialmente en contextos educativos.

I-A. Objetivo General

Diseñar filtros digitales FIR en Python usando scipy.signal, aplicarlos a señales I/Q capturadas del ADALM-Pluto, visualizar respuestas en frecuencia, y validar mediante filtrado de tonos y señales reales.

I-B. Objetivos Específicos

1. Diseñar filtros FIR con scipy.signal.firwin para paso bajo, paso alto y paso banda
2. Calcular y graficar respuesta en frecuencia de filtros diseñados
3. Capturar señales I/Q del Pluto con múltiples tonos
4. Aplicar filtros diseñados a señales capturadas
5. Validar mediante FFT: verificar atenuación de frecuencias no deseadas
6. Comparar filtros con diferentes órdenes (32, 64, 128 taps)

El documento se organiza de la siguiente manera: la Sección II presenta el marco teórico de filtros FIR y SDR, la Sección III detalla la metodología y especificaciones técnicas, la Sección IV muestra la implementación en Python, la Sección V presenta los resultados y análisis, y finalmente la Sección VI expone las conclusiones.

II. MARCO TEÓRICO

II-A. Filtros Digitales FIR

Un filtro digital FIR es un sistema discreto cuya respuesta al impulso tiene duración finita. La relación entrada-salida está dada por la ecuación de convolución:

$$y[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n-k] \tag{1}$$

donde $x[n]$ es la entrada, $y[n]$ es la salida, $h[k]$ son los coeficientes del filtro (respuesta al impulso), y M es el número de coeficientes o taps.

La función de transferencia en el dominio Z es:

$$H(z) = \sum_{k=0}^{M-1} h[k] \cdot z^{-k} \tag{2}$$

II-B. Propiedades de los Filtros FIR

Los filtros FIR presentan características fundamentales para aplicaciones SDR:

- **Estabilidad inherente**: Al no tener polos fuera del círculo unitario, son siempre estables.
- **Fase lineal**: Si los coeficientes son simétricos o anti-simétricos, cumplen:

$$h[n] = \pm h[M-1-n] \tag{3}$$

  Esto evita distorsión por fase en señales moduladas.
- **Implementación eficiente**: La estructura de convolución directa permite procesamiento en tiempo real.

II-C. Método de Ventanas para Diseño FIR

El método de ventanas es una técnica simple y efectiva para diseñar filtros FIR. Los pasos son:

1. Definir la respuesta ideal deseada $H_d(e^{j\omega})$
2. Calcular la respuesta al impulso ideal $h_d[n]$ mediante la transformada inversa de Fourier
3. Aplicar una ventana $w[n]$ para truncar y suavizar:

$$h[n] = h_d[n] \cdot w[n] \tag{4}$$

La ventana de Hamming utilizada en este proyecto se define como:

$$w[n] = 0{,}54 - 0{,}46 \cos\left(\frac{2\pi n}{M-1}\right), \quad 0 \leq n \leq M-1 \tag{5}$$

Sus características de rizado en banda de paso (0.02 dB) y atenuación en banda de rechazo (53 dB) la hacen adecuada para aplicaciones de telecomunicaciones.

II-D. Señales I/Q en SDR

La representación en fase y cuadratura (I/Q) permite describir señales moduladas mediante componentes en fase (I) y en cuadratura (Q):

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t) \tag{6}$$

Equivalentemente, en representación compleja:

$$s(t) = \Re\left\{x(t)e^{j2\pi f_c t}\right\}, \quad x(t) = I(t) + jQ(t) \tag{7}$$

La señal $x(t)$ se denomina envolvente compleja y contiene toda la información de banda base.

II-E. ADALM-Pluto: Características y Configuración

El ADALM-Pluto es un SDR activo que opera en el rango de 325 MHz a 3.8 GHz. Sus parámetros configurables incluyen:

Cuadro I
ESPECIFICACIONES TÉCNICAS DEL ADALM-PLUTO

Parámetro	**Valor**
Rango de frecuencia	325 MHz - 3.8 GHz
Ancho de banda instantáneo	20 MHz (configurable hasta 56 MHz)
Resolución ADC/DAC	12 bits
Frecuencia de muestreo máxima	61.44 MSPS
Potencia de salida	7 dBm (típica)
Ruido de fase	-90 dBc/Hz @ 1 kHz

II-F. Transformada Rápida de Fourier (FFT)

La FFT es el algoritmo eficiente para calcular la DFT (Discrete Fourier Transform):

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1 \tag{8}$$

Para análisis espectral, la densidad espectral de potencia se calcula como:

$$S_{xx}[k] = \frac{|X[k]|^2}{N \cdot f_s} \tag{9}$$

II-G. Convolución Discreta

La convolución en tiempo discreto se implementa eficientemente mediante:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k] \tag{10}$$

En procesamiento de señales SDR, la convolución se aplica directamente a los datos I/Q capturados.

III. METODOLOGÍA

III-A. Arquitectura del Sistema

La arquitectura propuesta consta de tres módulos principales:

1. **Módulo de diseño**: Implementación de filtros FIR mediante scipy.signal
2. **Módulo de captura**: Adquisición de señales I/Q del ADALM-Pluto
3. **Módulo de procesamiento**: Aplicación de filtros y análisis espectral

El flujo de datos se ilustra conceptualmente en la Figura 1.

Generador → Pluto TX → Medio → Pluto RX → Filtro FIR → FFT

Figura 1. Diagrama de flujo del sistema experimental

III-B. Especificaciones Técnicas de los Filtros

Los filtros diseñados deben cumplir con los siguientes requisitos:

Cuadro II
ESPECIFICACIONES DE DISEÑO DE FILTROS FIR

Tipo	Banda de paso	Banda de rechazo	Atenuación mínima
Pasa-bajas	0 - 100 kHz	¿120 kHz	40 dB
Pasa-altas	200 kHz - 1 MHz	¡180 kHz	40 dB
Pasa-banda	150 - 250 kHz	¡130 kHz y ¿270 kHz	40 dB

La frecuencia de muestreo seleccionada es $f_s = 2$ MHz, cumpliendo el teorema de Nyquist para señales de hasta 1 MHz.

III-C. Decisiones de Diseño Justificadas

- **Ventana de Hamming**: Seleccionada por su óptimo balance entre atenuación en banda de rechazo (53 dB) y ancho de transición.
- **Número de taps = 64**: Ofrece una transición de aproximadamente $\Delta f = 4f_s/M = 125$ kHz, adecuada para las especificaciones.
- **Frecuencia de muestreo de 2 MHz**: Permite procesar señales de hasta 1 MHz con un factor de sobremuestreo suficiente.

III-D. Señales de Prueba

Para validar el comportamiento de los filtros, se generan tres tipos de señales de prueba:

III-D1. Señal Multi-tono: Compuesta por tres tonos senoidales con frecuencias estratégicamente seleccionadas:

$$x_{\text{multi}}(t) = \sum_{i=1}^{3} A_i \cos(2\pi f_i t + \phi_i) \tag{11}$$

Con parámetros:

- $f_1 = 50$ kHz, $A_1 = 1{,}0$ (dentro de rechazo de pasa-altas)
- $f_2 = 200$ kHz, $A_2 = 1{,}0$ (frecuencia de referencia)
- $f_3 = 300$ kHz, $A_3 = 1{,}0$ (dentro de rechazo de pasa-bajas)

III-D2. Señal Chirp (Barrido lineal): Señal cuya frecuencia varía linealmente en el tiempo:

$$x_{\text{chirp}}(t) = \cos\left(2\pi\left(f_0 t + \frac{B}{2T}t^2\right)\right), \quad 0 \le t \le T \tag{12}$$

Donde $f_0 = 10$ kHz, $B = 490$ kHz (barrido hasta 500 kHz), $T = 1$ ms.

III-D3. Señal QPSK Simulada: Señal modulada digitalmente con 4 estados de fase:

$$s_{\text{QPSK}}(t) = \sum_k \left[I_k \cdot p(t - kT_s)\cos(2\pi f_c t) - Q_k \cdot p(t - kT_s)\sin(2\pi f_c t)\right] \tag{13}$$

Donde:

- $T_s = 5$ μs (tasa de 200 kbaudios)
- $(I_k, Q_k) \in \{(\pm 1, \pm 1)\}$ (símbolos QPSK)
- $p(t)$: pulso rectangular de duración T_s

III-E. Métricas de Evaluación

Para cuantificar el rendimiento de los filtros, se definen las siguientes métricas:

Cuadro III
MÉTRICAS DE EVALUACIÓN PARA FILTROS FIR

Métrica	Fórmula	Objetivo
Atenuación en rechazo	$A_{\text{stop}} = -20\log_{10}(\lvert H(f_{\text{stop}})\rvert)$	> 40 dB
Rizado en banda de paso	$R_p = \text{máx}(\lvert H(f)\rvert) - \text{mín}(\lvert H(f)\rvert)$	< 1 dB
Ancho de transición	$\Delta f = f_{\text{stop}} - f_{\text{pass}}$	< 50 kHz
Error cuadrático medio	$ECM = \frac{1}{N}\sum(y_{\text{ideal}} - y_{\text{real}})^2$	Minimizar

III-F. Herramientas y Flujo de Trabajo

Las herramientas utilizadas en la implementación son:

Cuadro IV
HERRAMIENTAS DE SOFTWARE UTILIZADAS

Herramienta	Versión	Propósito
Python	3.9+	Lenguaje principal
NumPy	1.21+	Operaciones matriciales
SciPy	1.7+	Diseño de filtros
Matplotlib	3.4+	Visualización
pyadi-iio	1.0+	Control del Pluto
Jupyter Lab	3.0+	Desarrollo interactivo

El flujo de trabajo experimental se detalla a continuación:

1. Diseño de coeficientes FIR con scipy.signal.firwin
2. Verificación mediante respuesta en frecuencia
3. Generación de señal multi-tono en Python
4. Transmisión por Pluto a frecuencia portadora de 2.45 GHz
5. Captura de señal recibida (2000 muestras)
6. Aplicación de filtros mediante convolución
7. Análisis espectral con FFT (ventana de Hanning)
8. Comparación con simulación ideal
9. Iteración para optimización (32, 64, 128 taps)

III-G. Cronograma de Actividades

El proyecto se desarrolló según el siguiente cronograma:

Cuadro V
CRONOGRAMA DE ACTIVIDADES DEL PROYECTO

Actividad	Fecha inicio	Fecha fin
Revisión bibliográfica y marco teórico	10/04/2026	12/04/2026
Diseño de filtros en Python	12/04/2026	14/04/2026
Configuración y pruebas con Pluto	14/04/2026	15/04/2026
Generación de señales de prueba	15/04/2026	16/04/2026
Captura y procesamiento de datos	16/04/2026	17/04/2026
Análisis de resultados	17/04/2026	17/04/2026
Elaboración del informe final	17/04/2026	17/04/2026

III-H. Generación y Transmisión de Señal Multi-tono
III-I. Captura y Aplicación de Filtros
III-J. Visualización de Resultados con Matplotlib
III-K. Comparación de Órdenes de Filtros (32, 64, 128 taps)
IV. RESULTADOS Y ANÁLISIS
IV-A. Respuesta en Frecuencia de los Filtros
IV-B. Resultados con Señal Multi-tono
IV-C. Comparación de Órdenes de Filtros
IV-D. Análisis Espectral con FFT
IV-E. Validación Experimental
IV-F. Discusión de Discrepancias
V. CONCLUSIONES
V-A. Síntesis de Logros
V-B. Limitaciones Encontradas
V-C. Lecciones Aprendidas
V-D. Trabajo Futuro
REFERENCIAS
REFERENCIAS
ANEXOS
A. Código Python completo
B. Script de diseño de filtros (fir_design.py)
C. Script de transmisión y captura con Pluto (pluto_fir.py)
D. Script de comparación de órdenes (compare_orders.py)
E. Gráficas de respuestas en frecuencia y espectros
F. Enlace al repositorio GitHub
https://github.com/telecom-unach/fir-filters-pluto
REFERENCIAS